OnnxRuntime User Guide

版本：0.1

发布日期：2026-11-06

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称"Amlogic"）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址：www.amlogic.cn
- 售前咨询：contact@amlogic.com
- 技术支持：support@amlogic.com

修订记录

版本	变更描述
0.1 (2026-11-06)	初始版本。

目录

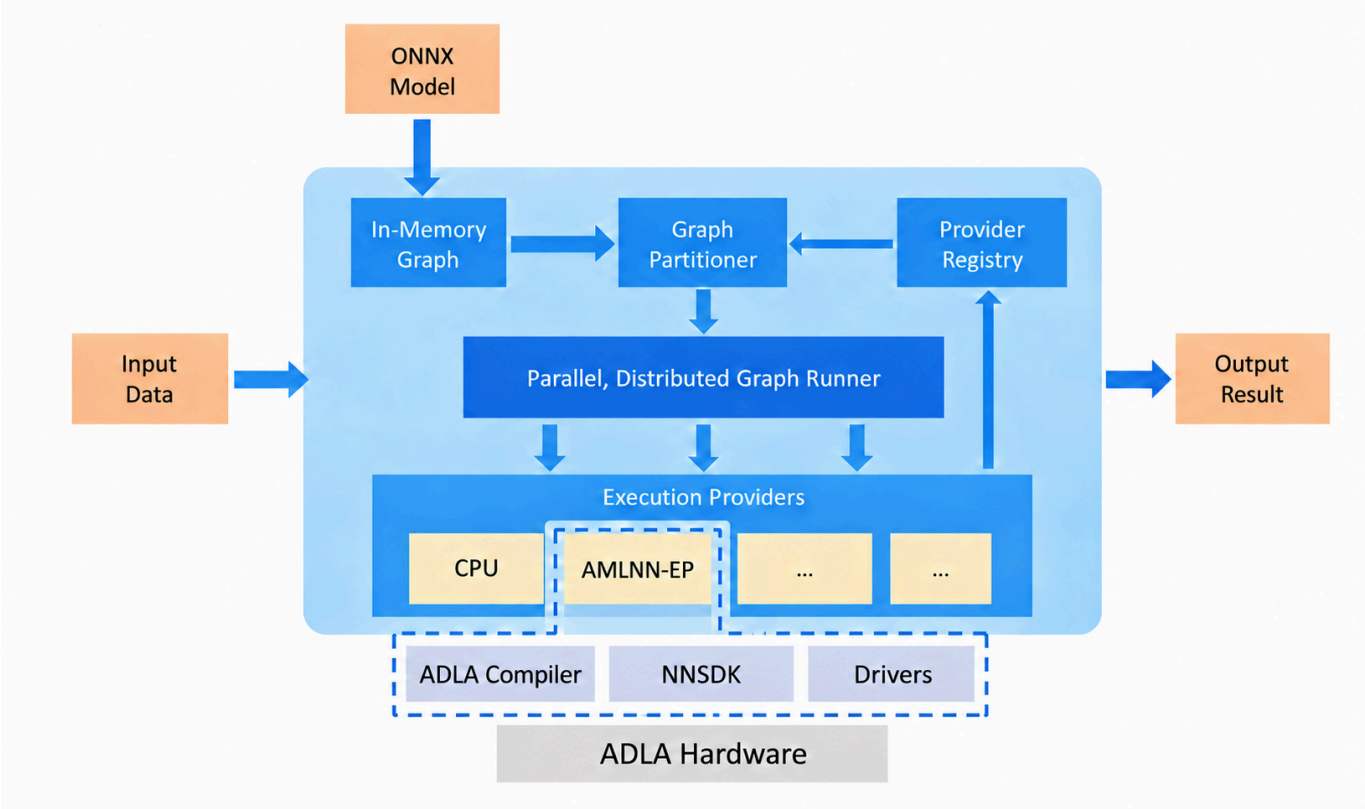
1 OnnxRuntime简介
  1.1 OnnxRuntime workflow
  1.2 平台支持
2 开发环境确认
  2.1 版本确认
    2.1.1 Driver版本确认
    2.1.2 Runtime版本确认
  2.2 系统环境
  2.3 平台类型确认
  2.4 快速验证
3 OnnxRuntime C/C++部署
  3.1 安装编译工具链
    3.1.1 安装 CMake 与主机工具
    3.1.2 安装交叉编译工具链
      3.1.2.1 Android系统：安装与配置 NDK
      3.1.2.2 Buildroot系统：安装 GCC 交叉编译工具链
      3.1.2.3 Armbian/Yocto系统：安装 Poky SDK
  3.2 OnnxRuntime编译运行
    3.2.1 源码下载
    3.2.2 编译运行
      3.2.2.1 Android系统编译运行
      3.2.2.2 Buildroot系统编译运行
      3.2.2.3 Armbian/Yocto系统编译运行
4 支持模型
5 FAQ

1 OnnxRuntime简介

本指南为 Amlogic NPU 平台 ONNX Runtime 部署使用文档，帮助开发者快速完成环境搭建、模型转换、推理调用与性能验证。基于标准 ONNX Runtime 架构扩展了 amlnn 硬件加速执行提供程序，可直接在指定芯片平台上加载 ONNX 模型，充分释放 NPU 算力，降低端侧模型的部署门槛。文档提供完整的安装步骤、API 示例、常见问题排查方法，助力开发者快速完成 AI 应用落地。

ONNX Runtime是微软主导开发的开源跨平台高性能机器学习推理与训练加速引擎，专为执行符合ONNX标准格式的模型设计，是工业级AI模型部署的主流方案。工程链接：https://github.com/microsoft/onnxruntime

1.1 OnnxRuntime workflow



整体处理流程如下：

Step 1：应用程序将浮点 ONNX 模型和输入张量提交给 ONNX Runtime。
Step 2：ONNX Runtime 解析模型，并将支持的计算图交给 AMLNN EP。
Step 3：AMLNN EP 调用 Core/Compiler，在目标设备上将计算图在线转换为 ADLA。
Step 4：NNSDK 加载生成的 ADLA，并调用 NPU 执行推理。
Step 5：推理结果经 AMLNN EP 返回 ONNX Runtime，最终输出给应用程序。

模型转换和推理均在目标设备上完成，无需手动搬运生成的 ADLA 文件。

1.2 平台支持

本文档适配的产品如下：

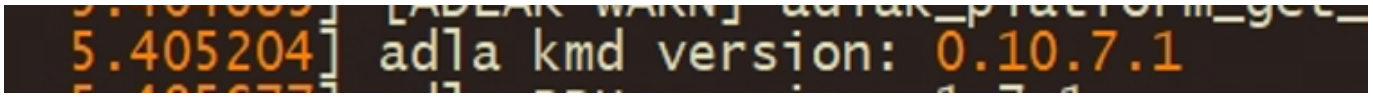
- A311Y3

2 开发环境确认

2.1 版本确认

2.1.1 Driver版本确认

Driver版本确认：重启单板，在串口或者在CMD窗口adb shell连接后，执行命令：dmesg |grep adla ，获取到类似如下log：



NPU Driver 版本为：0.10.7 (获取前三位对应的版本号即可)

当前文档内容支持NPU Driver版本>=2.0.0以上的版本。

2.1.2 Runtime版本确认

本版本运行链路包含以下组件。表中的版本分别对应 Core API、ADLA Compiler 和 NNSDK2 运行库，三者含义不同，不可相互替代。

组件	本版本	确认方式
`aml_compiler_core`	1.0.2	以厂商发布说明为准；程序可调用公开接口 `AmlCompilerCoreGetVersion()` 查询实际加载库的版本。
ADLA Compiler（`libcompiler_adla.so`）	3.5.3	在线转换日志应显示 `Compiler Config: Compiler Version: 3.5.3`。
NNSDK2	v1.1.0	在目标设备执行下方与当前系统对应的命令，读取板端 `libnnsdk.so` 的版本信息；命令输出中的 NNSDK2 版本应为 `v1.1.0`。

不同系统的 `libnnsdk.so` 安装路径不同，请在目标设备上选择对应的命令执行：

```
# Android系统
strings /vendor/lib64/libnnsdk.so | grep NNSDK

# Armbian/Yocto系统 或 Buildroot系统
strings /usr/lib/libnnsdk.so | grep NNSDK
```

如果命令未输出版本信息，应先检查 `libnnsdk.so` 是否存在于指定路径。板端使用的 `libnnsdk.so` 必须与目标系统及位宽匹配。 `AmlCompilerCoreGetVersion()` 查询的是进程实际加载的 `libaml_compiler_core.so`。`libcompiler_adla.so` 的版本则以对应厂商发布说明及在线转换日志为准。不得混用 Android、Buildroot 和 Armbian/Yocto 的库，也不得混用 32 位与 64 位库。

2.2 系统环境

确认板端系统环境与架构，旨在为后续章节选取对应系统和位宽的**运行库**及**交叉编译工具链**提供指导。

Step1：先确认是Android平台或者Linux平台

执行命令：adb shell getprop ro.product.cpu.abi

判定逻辑：

- **输出** `arm64-v8a` : 则确认为 64 位 Android 系统
- **输出** `armeabi-v7a` : 则确认为32 位 Android 系统
- **报错** `command not found`：判定为 Linux 系统，请继续执行 Step 2

Step2：Linux 系统（Buildroot / Yocto / Armbian）环境进一步确认

Step2.1：确认是否为 Armbian / Yocto 系统

执行命令：cat /version.txt

参考输出示例(以 Armbian 为例)

```
imagename:a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/ArmBian-PB#786
SPIN=Linux/Debian/ArmBian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul  7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

判定逻辑：

- **有该文件存在:**则判断为armbian/Yocto系统，再通过Compiler版本信息判断是32位/64位系统。参考示例对应Armbian 64-bit (AArch64)**系统
- **文件不存在:** 命令执行失败，判断为Buildroot系统，请继续执行Step2.2

> **注意**：上述确认方式仅适用于 Amlogic 官方 Release 版本单板。第三方开发板请自行确认是否支持相同的版本文件与判定逻辑。

Step2.2 确认 Buildroot 系统架构

执行命令：adb shell uname -m

判定逻辑：

- `aarch64` : 则确认为 64 位 buildroot系统
- `armv7l` : 则确认为 32 位 buildroot系统

2.3 平台类型确认

当前采用 ONNX Runtime 部署方式，支持 A311Y3 平台，需事先确认当前平台是否确为 A311Y3。校验通过以下两种相互独立的方式完成，所有校验结果均可对照下方**《Part_number / Serial 对应关系表》**进行核对。对应关系表如下：

Part_number	Serial
A311Y3	Serial: **4d0a**01020000000070e84a

方式一：通过 Part_number 快速确认

直接读取单板硬件表面刻印的Part_number标识，确认是A311Y3即可。

方式二：通过 Serial 号校验确认

获取 Serial 信息：单板正常启动后，通过 ADB 连接设备，执行以下命令可读取 Serial 号：

```
adb shell cat /proc/cpuinfo
# 或
adb shell cat /proc/cpu_chipid
```

匹配规则：

- **首选规则**：优先截取Serial号的**前4位字符**，对照对应关系表完成匹配。
- **备选规则**：若前4位无匹配结果，可降级截取Serial号的前**3位字符**完成匹配 。

2.4 快速验证

Step 1：获取 native case

按目标系统下载对应的 **64 位** native case，三个包不可互换：

目标系统	下载地址
Android系统	onnxruntime_android64.tar.gz
Buildroot系统	onnxruntime_buildroot64.tar.gz
Armbian/Yocto系统	onnxruntime_armbian_yocto64.tar.gz

Step 2：运行 native case

在开发主机下载并解压对应资源，将完整测试目录传到匹配的目标设备，然后按照 **包内** `Readme.txt` 执行测试并判断结果。设备须具备兼容的 `libnnsdk.so`、NPU 驱动 和运行权限。native case 用于快速确认设备运行链路；32 位目标应按照第 3 章对应的 ABI 步骤构建与测试。

3 OnnxRuntime C/C++部署

3.1 安装编译工具链

3.1.1 安装 CMake 与主机工具

推荐使用 Ubuntu 22.04/24.04 `x86_64` 开发主机。安装公共构建工具：

```
sudo apt-get update
sudo apt-get install -y \
  build-essential binutils file git git-lfs curl unzip zip rsync openssh-
client \
  python3 python3-venv python3-pip ninja-build pkg-config
```

ONNX Runtime 1.31.0 配置阶段需要 CMake 3.28 或更高版本。如果系统包版本不足，使用独立 Python 虚拟环境安装 CMake，不要覆盖系统 CMake：

```
python3 -m venv "$REPO_ROOT/.venv-build"
source "$REPO_ROOT/.venv-build/bin/activate"
python -m pip install --upgrade pip
python -m pip install 'cmake>=3.28' ninja packaging
```

Android 交叉编译还需要 Android SDK 和 NDK r27c，安装方法见 3.1.2.1 节。

构建脚本默认使用源码中已包含的公开头文件；使用其他兼容头文件时，可设置 `AMLNN_NNSDK2_INCLUDE_DIR` 和 `AMLNN_AML_COMPILER_CORE_INCLUDE_DIR`。

3.1.2 安装交叉编译工具链

以下操作均在 Linux `x86_64` 开发机进行。工具链必须与目标系统的 CPU、glibc、libstdc++ 和 ABI 匹配。下文所列版本为本项目测试基线；若目标板 Release 明确指定版本， 应以目标板 Release 为准。

3.1.2.1 Android系统：安装与配置 NDK

从 Android 官方命令行工具 取得 `sdkmanager`，按 NDK r27c 官方发布页 固定版本 `27.2.12479018`。如果已有独立解压的 `android-ndk-r27c`，可直接将 `ANDROID_NDK_PATH` 指向它；否则在已安装命令行工具的 SDK 目录执行：

```
export ANDROID_SDK_PATH=/path/to/android-sdk
SDKMANAGER="$ANDROID_SDK_PATH/cmdline-tools/latest/bin/sdkmanager"
"$SDKMANAGER" --sdk_root="$ANDROID_SDK_PATH" --install \
  'ndk;27.2.12479018' 'platform-tools'
export ANDROID_NDK_PATH="$ANDROID_SDK_PATH/ndk/27.2.12479018"
```

Android 32 位和 64 位共用一套 NDK r27c，无需分别安装；两个 ABI 均使用 Android API 21：

位宽	ABI	NDK C++ 编译器入口
64 位	`arm64-v8a`	`aarch64-linux-android21-clang++`
32 位	`armeabi-v7a`	`armv7a-linux-androideabi21-clang++`

设备需支持对应 ABI，并在 `/vendor/lib64` 或 `/vendor/lib` 提供兼容的 `libnnsdk.so`。Android SDK 与 NDK 安装在开发机；连接设备的 ADB 服务器可以是另一台机器。

3.1.2.2 Buildroot系统：安装 GCC 交叉编译工具链

优先使用目标板 Release 指定的交叉工具链。本项目 32/64 位验证基线均为 Arm GNU Toolchain 13.2.Rel1 的 Linux `x86_64` 主机版。**不要下载 `*-none-elf` 裸机工具链。**

位宽	工具链目标	后续构建使用的变量
64 位	`aarch64-none-linux-gnu`	`BUILDROOT64_PATH`
32 位	`arm-none-linux-gnueabihf`	`BUILDROOT32_PATH`

按目标位宽下载对应压缩包并核对发布方校验值。先准备安装目录：

```
TOOLCHAIN_PARENT=/path/to/toolchains
mkdir -p "$TOOLCHAIN_PARENT"
```

64 位（aarch64）

```
tar -xf /path/to/arm-gnu-toolchain-13.2.rel1-x86_64-aarch64-none-linux-
gnu.tar.xz -C "$TOOLCHAIN_PARENT"
```

32 位（ARMv7）

```
tar -xf /path/to/arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-
gnueabihf.tar.xz -C "$TOOLCHAIN_PARENT"
```

仅执行目标位宽对应的解压命令。上述版本为本项目验证基线；若 Buildroot 目标板使用 其他版本的 glibc 或 libstdc++，应改用与其 Release 匹配的工具链，并确认板端能够提供 生成程序所需的符号版本。

3.1.2.3 Armbian/Yocto系统：安装 Poky SDK

必须从目标板厂商/BSP Release 获取对应 Poky SDK 安装包， 不能把上述 Arm GNU 或 Android NDK 当作 Yocto SDK。本项目使用 Poky SDK 4.0.20， 对应 Release 的安装包参考链接（下载后仍须核对目标板 Release 与 SDK 版本）：

- 64 位 Poky SDK 4.0.20

- 32 位 Poky SDK 4.0.20

仅运行目标位宽对应的安装命令，两个 SDK 须安装到不同的新目录：

```
# 64 位
export YOCTO64_PATH=/path/to/new/yocto64-sdk
sh /path/to/yocto64-sdk-installer.sh -y -d "$YOCTO64_PATH"

# 32 位
export YOCTO32_PATH=/path/to/new/yocto32-sdk
sh /path/to/yocto32-sdk-installer.sh -y -d "$YOCTO32_PATH"
```

32 位环境文件为 `environment-setup-armv7at2hf-neon-poky-linux-gnueabi`， 64 位为 `environment-setup-armv8a-poky-linux`。安装器须正常完成路径重定位；如果厂商安装包不支持 `-y -d`，按其安装说明指定独立目录。 `yocto/{32,64}/build_so.sh` 会自行加载环境文件，调用前不要手工 `source`。

3.2 OnnxRuntime编译运行

3.2.1 源码下载

源码下载：aml-onnxruntime

编译使用指导、Demo 下载入口：amlnn-third-party-runtime/onnxruntime

Library 获取路径：amlnn-toolkit/amlnn_runtime/nn_runtime

按以下顺序获取源码、指导工具包和厂商库，再选择目标系统和位宽进行构建。

准备步骤一：命令执行约定

- **执行位置**：除标题明确写"板端"外，其余命令均在 Linux `x86_64` 开发主机执行；板端命令在目标设备执行。
- **脚本启动**：所有 `.sh` 构建脚本均使用 `bash path/to/script.sh` 启动，**不要使用 `source script.sh` 或 `. script.sh`**，以免脚本影响当前交互 Shell。
- **占位符**：文中的 `/path/to/...`、`board-address` 必须先替换为实际路径或地址，不要原样执行；构建和打包脚本会报告缺失的输入。
- **构建目录**：不同平台、ABI 和转换器配置必须使用不同的 `BUILD_DIR`（脚本默认目录已按平台分开），不要把 Android 与 Linux 交叉构建指向同一目录。

准备步骤二：获取源码

```
git clone -b amlogic/aml-npu-support \
  https://github.com/Amlogic-NN/aml-onnxruntime.git aml-onnxruntime
export REPO_ROOT="$(cd aml-onnxruntime && pwd -P)"
git -C "$REPO_ROOT" checkout --detach
08a78e6581a877aeab5d7036ca6d45c9cfdefc5f
git -C "$REPO_ROOT" submodule update --init --recursive
```

准备步骤三：获取并放置 AMLNN Release Tools

从指导仓库获取整个 `onnxruntime/` 目录，放入源码树的 `tools/amlnn_release/`：

```
cd "$(dirname "$REPO_ROOT")"
git clone https://github.com/Amlogic-NN/amlnn-third-party-runtime.git
(
  set -e
  test ! -e "$REPO_ROOT/tools/amlnn_release" && \
    test ! -L "$REPO_ROOT/tools/amlnn_release"
  cp -a amlnn-third-party-runtime/onnxruntime \
    "$REPO_ROOT/tools/amlnn_release"
  bash "$REPO_ROOT/tools/amlnn_release/common/verify_source_tree.sh"
)
```

复制后目录结构如下：

```
aml-onnxruntime/
└── tools/amlnn_release/
    ├── android/{32,64}/
    ├── linux/{32,64}/
    ├── yocto/{32,64}/
    ├── common/
    └── pipeline_demo/
```

准备步骤四：获取运行库

amlnn-toolkit 的 `nn_runtime` 提供各平台 32 位和 64 位的 Core、Compiler 与 NNSDK 库。Core 与 Compiler 位于同一 ABI 目录，构建脚本可直接使用，无需重新排布。

```
cd "$(dirname "$REPO_ROOT")"
git lfs install
git clone https://github.com/Amlogic-NN/amlnn-toolkit.git
git -C amlnn-toolkit lfs pull --include='amlnn_runtime/nn_runtime/**'
export NN_RUNTIME_ROOT="$(cd amlnn-toolkit/amlnn_runtime/nn_runtime && pwd -
P)"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
```

下表路径均相对于 `NN_RUNTIME_ROOT`。Core/Compiler 目录同时包含 `libaml_compiler_core.so` 和 `libcompiler_adla.so`，NNSDK 库文件名为 `libnnsdk.so`。

目标平台	Core / Compiler 目录	NNSDK 目录
Android 64 位	`aml_compiler_core/libs/android/arm64-v8a`	`nnsdk/android/lib64`

目标平台	Core / Compiler 目录	NNSDK 目录
Android 32 位	aml_compiler_core/libs/android/armeabi-v7a	nnsdk/android/lib32
Buildroot 64 位	aml_compiler_core/libs/linux/aarch64	nnsdk/linux/buildroot/lib64
Buildroot 32 位	aml_compiler_core/libs/linux/armv7	nnsdk/linux/buildroot/lib32
Armbian/Yocto 64 位	aml_compiler_core/libs/yocto/aarch64	nnsdk/linux/yocto/lib64
Armbian/Yocto 32 位	aml_compiler_core/libs/yocto/armv7	nnsdk/linux/yocto/lib32

板端若已有兼容的 libnnsdk.so，优先使用板端版本；否则由设备集成方按目标系统 和位宽部署上述对应版本。不要混用不同平台或位宽的库。

按目标位宽依次构建共享库、Demo 和运行包，然后在板端运行。新开终端后，先设置以下通用 路径变量。Android 还需设置 3.1.2.1 节的 ANDROID_SDK_PATH 和 ANDROID_NDK_PATH；Buildroot 与 Armbian/Yocto 的工具链变量在各平台小节中设置。

```
export REPO_ROOT=/absolute/path/to/aml-onnxruntime
export NN_RUNTIME_ROOT=/absolute/path/to/amlnn-
toolkit/amlnn_runtime/nn_runtime
```

3.2.2 编译运行

3.2.2.1 Android系统编译运行

Step 1：构建共享库（32/64 位）

64 位（arm64-v8a）

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android64"
bash tools/amlnn_release/android/64/build_so.sh
```

32 位（armeabi-v7a）：使用 Android NDK r27c。设备应支持 armeabi-v7a， 并提供 /system/bin/linker 和 /vendor/lib/libnnsdk.so。

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
```

```
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android32"
bash tools/amlnn_release/android/32/build_so.sh
```

产物位于 `$BUILD_DIR/Release/libonnxruntime.so`。32/64 位不得共用构建目录；主机资源充足时可设置 `PARALLEL_JOBS=8`。

Step 2：构建 Demo 与运行包（32/64 位）

Demo 必须使用同平台、同位宽的 ORT 产物；下方命令已显式指定 `ORT_BUILD_DIR`。

64 位（arm64-v8a）

```
cd "$REPO_ROOT"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android64"
ORT_BUILD_DIR="$BUILD_DIR/Release" bash
tools/amlnn_release/android/64/build_demo.sh
```

Demo 产物位于 `tools/amlnn_release/pipeline_demo/bin/android64/`，其中包含 `pipeline_demo` 和 `libonnxruntime.so`。随后准备运行包。`AMLNN_VENDOR_SDK_ROOT` 须指向运行库根目录，输出包须使用源码树外的新目录。

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-android-package.XXXXXX)"
DEMO_PACKAGE_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/android64
"
PACKAGE_DIR="$DEMO_PACKAGE_DIR" OUTPUT_DIR="$PACKAGE_PARENT/android64" bash
tools/amlnn_release/android/64/prepare_package.sh
(cd "$PACKAGE_PARENT/android64" && sha256sum -c SHA256SUMS)
```

32 位（armeabi-v7a）：在新 Shell 中重设独立 `BUILD_DIR` 后执行：

```
cd "$REPO_ROOT"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android32"
ORT_BUILD_DIR="$BUILD_DIR/Release" bash
tools/amlnn_release/android/32/build_demo.sh
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-android-package.XXXXXX)"
DEMO_PACKAGE_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/android32
"
PACKAGE_DIR="$DEMO_PACKAGE_DIR" OUTPUT_DIR="$PACKAGE_PARENT/android32" bash
tools/amlnn_release/android/32/prepare_package.sh
(cd "$PACKAGE_PARENT/android32" && sha256sum -c SHA256SUMS)
```

Demo 位于 `tools/amlnn_release/pipeline_demo/bin/android32/`。

Step 3：板端运行（32/64 位）

先完成 3.2.2.1 节 Step 2 中**对应位宽**的 Demo 与运行包准备，再将完整包和 ONNX 模型传到 Android 设备。

部署要求

- 包内含 `pipeline_demo`、`libonnxruntime.so`、同 ABI 的 Core/Compiler、`libc++_shared.so`、`run_demo.sh`、批量脚本和 `SHA256SUMS`。
- 设备须提供同 ABI 的 NNSDK 与 NPU 驱动。
- 32 位与 64 位包不可混用。

在**设备终端**执行以下命令，按实际位宽选择 `/vendor/lib64` 或 `/vendor/lib`：

```
PACKAGE_DIR=/path/on/device/to/android64-or-android32
NNSDK_LIB_DIR=/vendor/lib64  # Android 32 位改为 /vendor/lib
MODEL=/path/on/device/to/model.onnx
OUTPUT_DIR=/path/on/device/to/new-output-directory
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

结果判定

- 日志级别可选 `error`（最少日志，**不是错误状态**）、`info`、`debug`；不支持 `warning`，传入会被程序明确拒绝。
- `OUTPUT_DIR` 必须尚不存在，父目录须可写；未提供输入文件时 Demo 使用零输入，仅验证链路。若需真实输入，在 `error` 后按模型输入顺序追加 `input0.bin` 等原始输入文件。
- `run_demo.sh` 会检查生成的非空 ADLA、`output_0.bin` 和 `output_0.meta`；输出形状、数据类型及数值精度须另行与模型参考结果核对。

可选：ADB 快捷运行：主机已连接 Android 设备时，可直接执行 `tools/amlnn_release/android/64/run_demo.sh`（32 位为 `android/32/run_demo.sh`）。参数顺序为 `模型路径 输出目录 [日志级别] [输入文件...]`。脚本会将 Demo、ORT、Core、Compiler、`libc++_shared.so` 和模型推送到设备临时目录并运行，NNSDK 分别从 `/vendor/lib64` 或 `/vendor/lib` 加载。默认使用 `ANDROID_NDK_PATH` 下的 `libc++_shared.so`；也可通过 `AMLNN_LIBCXX_SO` 显式指定。连接多台设备时，须先设置 `ANDROID_SERIAL`。

3.2.2.2 Buildroot系统编译运行

Step 1：构建共享库（32/64 位）

`linux/32` 与 `linux/64` 分别使用厂商 SDK 中 `linux/armv7` 和 `linux/aarch64` 下的成对库。交叉工具链必须与目标板的 glibc、libstdc++ 和 ABI 匹配，不要仅凭目录名判断兼容性。本项目 32/64 位验证基线均为 Arm GNU Toolchain 13.2.Rel1；`BUILDROOT{32,64}_PATH` 指向**已解压的工具链根目录**。

64 位（aarch64）

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILDROOT64_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-aarch64-
none-linux-gnu
```

```
export PATH="$BUILDROOT64_PATH/bin:$PATH"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot64"
export PARALLEL_JOBS=8
bash tools/amlnn_release/linux/64/build_so.sh
```

32 位（ARMv7）：使用独立 Shell、工具链及构建目录，不得复用 64 位 CMake 缓存。

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILDROOT32_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-linux-gnueabihf
export PATH="$BUILDROOT32_PATH/bin:$PATH"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot32"
export PARALLEL_JOBS=8
bash tools/amlnn_release/linux/32/build_so.sh
```

产物位于 `$BUILD_DIR/Release/libonnxruntime.so`；打包脚本会检查目标架构。

Step 2：构建 Demo 与运行包（32/64 位）

64 位（aarch64）

```
cd "$REPO_ROOT"
export BUILDROOT64_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-aarch64-
none-linux-gnu
export PATH="$BUILDROOT64_PATH/bin:$PATH"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot64"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/linux64"
ORT_BUILD_DIR="$BUILD_DIR/Release" OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/linux/64/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-linux64-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/linux64" bash
tools/amlnn_release/linux/64/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/linux64"
```

32 位（ARMv7）：在另一 Shell 中按 3.2.2.2 节 Step 1 设置 `BUILDROOT32_PATH` 和独立的 `BUILD_DIR`，然后执行：

```
cd "$REPO_ROOT"
export BUILDROOT32_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-linux-gnueabihf
export PATH="$BUILDROOT32_PATH/bin:$PATH"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot32"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/linux32"
ORT_BUILD_DIR="$BUILD_DIR/Release" OUTPUT_DIR="$DEMO_DIR" bash
```

```
tools/amlnn_release/linux/32/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-linux32-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/linux32" bash
tools/amlnn_release/linux/32/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/linux32"
```

说明

- 打包脚本用 `readelf` 检查 ONNX Runtime、Core、Compiler 和 Demo 的 ELF 位宽与架构，并输出 `SHA256SUMS`。
- 运行包不含目标板提供的 `libnnsdk.so`。

Step 3：板端运行（32/64 位）

先完成 3.2.2.2 节 Step 2 中**对应位宽**的 Demo 与运行包准备；Demo 与 ORT 库必须使用同一源码、交叉工具链和 ABI。

运行包内容（**不含**目标系统的 `libnnsdk.so`、NPU 驱动或模型）

- `pipeline_demo`、`run_demo.sh`、`run_batch.sh`
- `libonnxruntime.so`、`libonnxruntime.so.1`
- 同平台同位宽的 `libaml_compiler_core.so` 和 `libcompiler_adla.so`
- `SHA256SUMS`（打包脚本会检查三个共享库及 Demo 的 ELF 位宽与架构）

部署要求

- 将整个包通过 `tar`/`scp` 复制到对应位宽的目标板可执行目录，解包后在包目录执行 `sha256sum -c SHA256SUMS`。
- 32 位板必须使用 32 位包，64 位板必须使用 64 位包。
- 板端必须具备匹配的 glibc、libstdc++、NNSDK、NPU 驱动与权限，以及可读的 ONNX 模型和可写的输出目录。

板端执行：`NNSDK_LIB_DIR` 指向目标板已有的 `libnnsdk.so` 所在目录（例如 `/usr/lib`），不要复制其他平台库：

```
PACKAGE_DIR=/path/on/board/to/linux64-or-linux32
NNSDK_LIB_DIR=/usr/lib
MODEL=/path/on/board/to/model.onnx
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
OUTPUT_PARENT="$(mktemp -d /tmp/amlnn-smoke.XXXXXX)"
OUTPUT_DIR="$OUTPUT_PARENT/output"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

结果判定

- 另行核对输出形状、数据类型和参考输出；非零退出码、缺失 ADLA 或缺失输出文件均为失败。
- `run_demo.sh` 会创建新的输出目录，运行在线转换与推理，并校验非空 ADLA、`output_0.bin` 和 `output_0.meta`；已存在的输出目录会被拒绝，以免覆盖既有结果。

批量测试：脚本仅处理模型目录当前层级中的 `.onnx` 文件，并为每个模型单独启动 进程。脚本不会删除模型或中间产物；模型失败时记录日志，并继续处理其余模型。

```
MODEL_DIR=/path/on/board/to/onnx-models
BATCH_PARENT="$(mktemp -d /tmp/amlnn-batch.XXXXXX)"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_batch.sh" "$MODEL_DIR"
"$BATCH_PARENT/results"
# 查看 $BATCH_PARENT/results/manifest.tsv 和各模型的 .log 文件
```

所有模型成功时退出码为 `0`，有失败模型时为 `1`；结果根目录必须事先不存在。

3.2.2.3 Armbian/Yocto系统编译运行

Step 1：构建共享库（32/64 位）

构建脚本会自行加载 Poky SDK，调用前不要手工 `source`；每个位宽使用独立的 `BUILD_DIR`。

64 位（aarch64）：使用与目标系统匹配的 Poky SDK；安装方法见 3.1.2.3 节。

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export YOCTO64_PATH=/path/to/installed/amlogic-yocto64-sdk
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto64"
export PARALLEL_JOBS=8  # 内存不足时改为 1
env -u LD_LIBRARY_PATH bash tools/amlnn_release/yocto/64/build_so.sh
```

产物位于 `$BUILD_DIR/Release/`，不能用 Linux/Buildroot 的库替代。

32 位（ARMv7）：使用与目标系统匹配的厂商 Poky/Yocto32 SDK，并按照 3.1.2.3 节安装。本项目验证基线为 SDK 4.0.20 和 `arm-poky-linux-gnueabi-g++` 11.4.0。

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export YOCTO32_PATH=/path/to/installed/amlogic-yocto32-sdk
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto32"
export PARALLEL_JOBS=8  # 内存不足时改为 1
env -u LD_LIBRARY_PATH bash tools/amlnn_release/yocto/32/build_so.sh
```

产物位于 `$BUILD_DIR/Release/`。

Step 2：构建 Demo 与运行包（32/64 位）

配套运行库从 `aml_compiler_core/libs/yocto/{aarch64,armv7}` 选择；Core 与 Compiler 位于同一 ABI 目录，不得换成 Linux/Buildroot 库。

64 位（aarch64）：在构建机用同一 Yocto64 SDK 交叉编译 Demo，并制作完整运行包。`BUILD_DIR` 必须与 3.2.2.3 节 Step 1 构建 `.so` 时相同，`OUTPUT_DIR` 为尚不存在的包目录：

```
cd "$REPO_ROOT"
export YOCTO64_PATH=/path/to/installed/amlogic-yocto64-sdk
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto64"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/yocto64"
env -u LD_LIBRARY_PATH OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/yocto/64/build_demo_cross.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-yocto64-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/yocto64" bash
tools/amlnn_release/yocto/64/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/yocto64"
```

32 位（ARMv7）：不要预先 `source` SDK：

```
cd "$REPO_ROOT"
export YOCTO32_PATH=/path/to/installed/amlogic-yocto32-sdk
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto32"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/yocto32"
env -u LD_LIBRARY_PATH OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/yocto/32/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-yocto32-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/yocto32" bash
tools/amlnn_release/yocto/32/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/yocto32"
```

说明：打包时须同时指定 `PIPELINE_DEMO`、`ORT_BUILD_DIR` 和新的 `OUTPUT_DIR`，不能只提供输出目录。

Step 3：板端运行（32/64 位）

先完成 3.2.2.3 节 Step 2 中**对应位宽**的 Demo 与运行包准备；Demo 与 ORT 库必须使用同一源码、厂商 Poky SDK 和 ABI。

运行包内容（**不含**目标系统的 `libnnsdk.so`、NPU 驱动或模型）

- `pipeline_demo`、`run_demo.sh`、`run_batch.sh`
- `libonnxruntime.so`、`libonnxruntime.so.1`
- 同平台同位宽的 `libaml_compiler_core.so` 和 `libcompiler_adla.so`
- `SHA256SUMS`（打包脚本会检查三个共享库及 Demo 的 ELF 位宽与架构）

部署要求

- 将整个包通过 `tar`/`scp` 复制到对应位宽的目标板可执行目录，解包后在包目录执行 `sha256sum -c SHA256SUMS`。
- 32 位板必须使用 32 位包，64 位板必须使用 64 位包。
- 板端必须具备匹配的 glibc、libstdc++、NNSDK、NPU 驱动与权限，以及可读的 ONNX 模型和可写的输出目录。

板端执行：`NNSDK_LIB_DIR` 指向目标板已有的 `libnnsdk.so` 所在目录（例如 `/usr/lib`），不要复制其他平台库：

```
PACKAGE_DIR=/path/on/board/to/yocto64-or-yocto32
NNSDK_LIB_DIR=/usr/lib
MODEL=/path/on/board/to/model.onnx
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
OUTPUT_PARENT="$(mktemp -d /tmp/amlnn-smoke.XXXXXX)"
OUTPUT_DIR="$OUTPUT_PARENT/output"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

结果判定

- 另行核对输出形状、数据类型和参考输出；非零退出码、缺失 ADLA 或缺失输出文件均为失败。
- `run_demo.sh` 会创建新的输出目录，运行在线转换与推理，并校验非空 ADLA、`output_0.bin` 和 `output_0.meta`；已存在的输出目录会被拒绝，以免覆盖既有结果。

批量测试：脚本仅处理模型目录当前层级中的 `.onnx` 文件，并为每个模型单独启动 进程。脚本不会删除模型或中间产物；模型失败时记录日志，并继续处理其余模型。

```
MODEL_DIR=/path/on/board/to/onnx-models
BATCH_PARENT="$(mktemp -d /tmp/amlnn-batch.XXXXXX)"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_batch.sh" "$MODEL_DIR"
"$BATCH_PARENT/results"
# 查看 $BATCH_PARENT/results/manifest.tsv 和各模型的 .log 文件
```

所有模型成功时退出码为 `0`，有失败模型时为 `1`；结果根目录必须事先不存在。

4 支持模型

下表按用途大类列出本版本支持的 ONNX 模型。

用途大类	模型系列	支持模型
图像分类与识别	AlexNet / CaffeNet	`bvlcalexnet-7.onnx` `caffenet-7.onnx`
	DenseNet / EfficientNet	`densenet-7.onnx` `efficientnet_b0.onnx`
	GoogLeNet / Inception	`googlenet-7.onnx` `inception-v1-7.onnx`
	MobileNet	`mobilenet_v2.onnx` `mobilenetv2-7.onnx`

	ResNet	`resnet18.onnx`、`resnet18-v1-7.onnx`、`resnet18-v2-7.onnx`
		`resnet34-v1-7.onnx`、`resnet34-v2-7.onnx`
		`resnet50-v1-12.onnx`、`resnet50-v1-7.onnx`、`resnet50-v2-7.onnx`
		`resnet101-v1-7.onnx`、`resnet101-v2-7.onnx`
		`resnet152-v1-7.onnx`、`resnet152-v2-7.onnx`

	ShuffleNet / SqueezeNet	`shufflenet-v2-10.onnx`
		`squeezenet1.0-12.onnx`
		`squeezenet1.1-7.onnx`

	VGG	`vgg16-7.onnx`
		`vgg19-7.onnx`
		`vgg19-bn-7.onnx`

	其他	`mnist-7.onnx`、`scene-middle.onnx`
		`age_googlenet.onnx`、`emotion-ferplus-7.onnx`
		`edge256_ped_reid_mbv1_20200815.onnx`

图像分割	Human Segmentation	`paddle_human_seg.onnx`
目标检测	YOLOv2	`yolov2-coco-9.onnx`
基础算子验证	Mul	`hpmul.onnx`

5 FAQ

现象	检查与处理
构建提示缺少 AMLNN Provider 或源码预检失败	核对源码提交版本和 `tools/amlnn_release/` 目录位置；上游原始 ONNX Runtime 不包含本项目接入。
CMake 生成器冲突	为不同生成器、系统和位宽使用新的 `BUILD_DIR`，不要复用旧缓存。
找不到 Core 或 Compiler	核对目标平台、ABI、同一发布包的成对库，以及包内 `SHA256SUMS`。
找不到 `libnnsdk.so`	检查目标设备提供的兼容 NNSDK，且 `NNSDK_LIB_DIR` 指向包含该文件的目录。
Android 找不到 `libc++_shared.so`	使用与构建所用 NDK 匹配的库，并确认它在完整运行包中。
Linux 加载器找不到 `libonnxruntime.so.1`	使用对应平台的 `prepare_package.sh` 打包；包内应含 SONAME 链接。
模型退出码为 `137`	结合系统日志确认是否遭到 OOM Killer；保留单模型日志并单独复现。
只有 `output_0.bin`，无法判断输出含义	同时读取 `output_0.meta` 中的名称、数据类型和形状；零输入结果不能代替数值验收。